holder获取方案文档

ytLp 协议 Holder 获取方案

本文档详细说明如何获取 ytLp 协议中三种角色的持有者（Holder）信息：YT 代币持有者、LP 代币持有者和 Lending 提供者。

目录

1. [YT 代币持有者](#1-yt-代币持有者)
2. [LP 代币持有者（ytLP）](#2-lp-代币持有者ytlp)
3. [Lending 提供者](#3-lending-提供者)
4. [完整解决方案](#3-完整解决方案示例)

---

1. YT 代币持有者

合约信息

合约: `YTAssetVault.sol`

位置: `contracts/ytVault/YTAssetVault.sol`

标准: ERC20（继承自 `ERC20Upgradeable`）

已部署的 YT 代币

代币名称	合约地址
YT-A	0x03d2a3B21238CD65D92c30A81b3f80d8bA1A44AC
YT-B	0xf41fc97d8a3c9006Dd50Afa04d0a3D6D27f8cD0B
YT-C	0xBdE54f062C537CA7D96Cd97e5B827918E38E97b5

获取方式

监听事件

YT 代币作为标准 ERC20 代币，会发出以下事件：

标准 ERC20 Transfer 事件:

```
代码块
event Transfer(address indexed from, address indexed to, uint256 value);
```

获取步骤:

1. 监听 `Transfer` 事件，收集所有接收过代币的地址
2. 过滤掉零地址 `0x0000...0000` （铸造）和合约地址本身（销毁）
3. 对于每个地址，调用 `balanceOf(address)` 查询当前余额
4. 余额 > 0 的地址即为持有者

---

2. LP 代币持有者（ytLP）

合约信息

合约: `YTLPToken.sol`

位置: `contracts/ytLp/tokens/YTLPToken.sol`

标准: ERC20（继承自 `ERC20Upgradeable`）

已部署地址: `0x102e3F25Ef0ad9b0695C8F2daF8A1262437eEfc3`

获取方式

监听事件

标准 ERC20 Transfer 事件:

```
代码块
event Transfer(address indexed from, address indexed to, uint256 value);
```

获取步骤:

1. 监听 ytLP 代币合约的 `Transfer` 事件
2. 收集所有接收过代币的地址（`to` 参数）
3. 过滤零地址和合约地址
4. 调用 `balanceOf(address)` 获取当前余额

5. 余额 > 0 的即为 LP 持有者

---

3. Lending 提供者

合约信息

合约: `Lending.sol`

位置: `contracts/ytLending/Lending.sol`

已部署地址: `0xCb4E7B1069F6C26A1c27523ce4c8dfD884552d1D`

特点: 非 ERC20 代币，通过 mapping 存储用户余额

数据存储结构

代码块

```
// 用户基本信息（在 LendingStorage.sol 中定义）
struct UserBasic {
    int104 principal;  // 本金（正数=存款本金，负数=借款本金）
}
mapping(address => UserBasic) public userBasic;

// 用户抵押品余额
mapping(address => mapping(address => uint256)) public userCollateral;
```

获取方式

监听事件

Lending 相关事件 (定义在 `ILending.sol`):

代码块

```
event Supply(address indexed from, address indexed dst, uint256 amount);
event Withdraw(address indexed src, address indexed to, uint256 amount);
event SupplyCollateral(address indexed from, address indexed dst, address indexed asset, uint256 amount);
event WithdrawCollateral(address indexed src, address indexed to, address indexed asset, uint256 amount);
event AbsorbDebt(address indexed absorber, address indexed borrower, uint256 basePaidOut, uint256 usdValue);
event AbsorbCollateral(address indexed absorber, address indexed borrower, address indexed asset, uint256 collateralAbsorbed, uint256 usdValue);
```

```
event BuyCollateral(address indexed buyer, address indexed asset, uint256 baseAmount, uint256 collateralAmount);
```

获取 USDC 提供者步骤:

1. 监听 `Supply` 事件，收集所有曾经提供 USDC 的地址（`dst` 参数）
2. 监听 `Withdraw` 事件，跟踪提现行为
3. 对每个地址调用 `supplyBalanceOf(address)` 查询当前供应余额
4. 供应余额 > 0 的即为当前的 Lending 提供者

获取抵押品提供者步骤:

1. 监听 `SupplyCollateral` 事件，收集提供抵押品的地址
2. 对每个地址和资产调用 `getCollateral(address, asset)` 查询余额
3. 抵押品余额 > 0 的即为抵押品提供者

余额说明:

- `principal` > 0: 用户是供应者（存款方）
- `principal` < 0: 用户是借款者
- `principal` = 0: 用户无供应也无借款
- 实际余额需要通过 `supplyBalanceOf()` 或 `borrowBalanceOf()` 获取（含利息累计）

注意事项

1. **利息累计**: Lending 使用复合式利息，余额会随时间增长

- `principal` 是存储的本金（不变）
- `supplyBalanceOf()` 返回实时余额（含利息）
- 需要调用 `accrueInterest()` 或读取函数自动累计利息

2. **借款和供应同时存在**:

- 一个地址可能既是供应者又是借款者
- `principal > 0`: 净供应者
- `principal < 0`: 净借款者

3. **抵押品种类**:

- 通过 `assetList()` 获取支持的抵押资产列表
- 对每个资产调用 `getCollateral(user, asset)` 查询余额

---

4. 完整解决方案示例

以下是一个完整的 TypeScript 脚本，可以获取所有三种 holder：

从合约部署区块开始扫描，扫描到当前区块后（记录扫描最终区块），每隔10s从上次记录的区块扫描到最新区块，以此类推，存储到数据库自行加代码。

代码块

```
import { ethers, Contract, JsonRpcProvider } from "ethers";
import type { EventLog, Log } from "ethers";

// ==================== 类型定义 ====================

interface VaultConfig {
  name: string;
  address: string;
}

interface YTHolderData {
  address: string;
  balance: string;
}

interface LPHolderData {
  address: string;
  balance: string;
  share: string;
}

interface LendingSupplierData {
  address: string;
  supply: string;
  borrow: string;
  net: string;
}

// ==================== 配置 ====================

const RPC_URL: string =
"https://api.zan.top/node/v1/arb/sepolia/baf84c429d284bb5b676cb8c9ca21c07";

// 合约配置（包含部署区块号，可以大幅减少查询时间）
const YT_VAULTS: VaultConfig[] = [
  { name: "YT-A", address: "0x97204190B35D9895a7a47aa7BaC61ac08De3cF05" },
  { name: "YT-B", address: "0x181ef4011c35C4a2Fda08eBC5Cf509Ef58E553fF" },
  { name: "YT-C", address: "0xE9A5b9f3a2Eda4358f81d4E2eF4f3280A664e5B0" },
];
```

```
39
40  const YTLP_ADDRESS: string = "0x102e3F25Ef0ad9b0695C8F2daF8A1262437eEfc3";
41  const LENDING_ADDRESS: string = "0xCb4E7B1069F6C26A1c27523ce4c8dfD884552d1D";
42
43  // ==================== 部署区块配置 ====================
44  //
45  // 配置说明:
46  // 1. 查询准确的部署区块号，直接填写
47
48  interface DeploymentConfig {
49    ytVaults: number;   // YT 代币部署区块
50    ytlp: number;       // ytLP 部署区块
51    lending: number;    // Lending 部署区块
52  }
53
54  const DEPLOYMENT_BLOCKS: DeploymentConfig = {
55    ytVaults: 227339300,  // YT-A/B/C 部署区块号
56    ytlp: 227230270,      // ytLP 部署区块号
57    lending: 227746053,   // Lending 部署区块号
58  };
59
60  // ==================== ABIs ====================
61
62  const ERC20_ABI = [
63    "event Transfer(address indexed from, address indexed to, uint256 value)",
64    "function balanceOf(address account) view returns (uint256)",
65    "function totalSupply() view returns (uint256)",
66  ] as const;
67
68  const LENDING_ABI = [
69    "event Supply(address indexed from, address indexed dst, uint256 amount)",
70    "function supplyBalanceOf(address account) view returns (uint256)",
71    "function borrowBalanceOf(address account) view returns (uint256)",
72  ] as const;
73
74  // ==================== 工具函数 ====================
75
76  /**
77   * 分块查询事件，避免超出 RPC 限制
78   * @param contract 合约实例
79   * @param filter 事件过滤器
80   * @param fromBlock 起始区块
81   * @param toBlock 结束区块
82   * @param batchSize 每批次的区块数量（默认 9999，低于 10000 限制）
83   */
84  async function queryEventsInBatches(
85    contract: Contract,
```

```
  filter: any,
  fromBlock: number,
  toBlock: number,
  batchSize: number = 9999
): Promise<(EventLog | Log)[]> {
  const allEvents: (EventLog | Log)[] = [];
  let currentBlock = fromBlock;

  console.log(`    查询区块范围: ${fromBlock} -> ${toBlock} (总共 ${toBlock - fromBlock + 1} 个区块)`);

  while (currentBlock <= toBlock) {
    const endBlock = Math.min(currentBlock + batchSize, toBlock);

    console.log(`    正在查询区块 ${currentBlock} - ${endBlock}...`);

    try {
      const events = await contract.queryFilter(filter, currentBlock, endBlock);
      allEvents.push(...events);
      console.log(`    ✓ 获取到 ${events.length} 个事件`);
    } catch (error) {
      console.error(`    ✗ 查询区块 ${currentBlock} - ${endBlock} 失败:`, error);
      throw error;
    }

    currentBlock = endBlock + 1;

    // 添加小延迟，避免触发 RPC 速率限制
    if (currentBlock <= toBlock) {
      await new Promise(resolve => setTimeout(resolve, 100));
    }
  }

  console.log(`    总计获取 ${allEvents.length} 个事件\n`);
  return allEvents;
}

/**
 * 获取当前最新区块号
 */
async function getLatestBlockNumber(provider: JsonRpcProvider, silent: boolean = false): Promise<number> {
  const blockNumber = await provider.getBlockNumber();
  if (!silent) {
    console.log(`当前最新区块: ${blockNumber}\n`);
  }
```

```
  return blockNumber;
}

// ==================== 主函数 ====================

// 记录上次扫描的区块号
let lastScannedBlock: number = 0;
// 标记是否正在扫描，防止并发
let isScanning: boolean = false;

// 全局地址集合，用于追踪所有曾经出现过的地址
const allYTAddresses: Map<string, Set<string>> = new Map(); // vault address -> holder addresses
const allLPAddresses: Set<string> = new Set();
const allLendingAddresses: Set<string> = new Set();

async function getAllHolders(
  provider: JsonRpcProvider,
  fromBlock?: number,
  toBlock?: number,
  isInitialScan: boolean = false
): Promise<void> {
  // 获取最新区块号
  const latestBlock = toBlock || await getLatestBlockNumber(provider, fromBlock !== undefined);

  // 计算起始区块
  let ytVaultsStartBlock: number;
  let ytlpStartBlock: number;
  let lendingStartBlock: number;

  if (fromBlock !== undefined) {
    // 增量扫描模式
    ytVaultsStartBlock = ytlpStartBlock = lendingStartBlock = fromBlock;
    console.log(`\n🔄 增量扫描: 区块 ${fromBlock} -> ${latestBlock}\n`);
  } else {
    // 首次扫描: 使用部署区块号
    ytVaultsStartBlock = DEPLOYMENT_BLOCKS.ytVaults;
    ytlpStartBlock = DEPLOYMENT_BLOCKS.ytlp;
    lendingStartBlock = DEPLOYMENT_BLOCKS.lending;
    if (isInitialScan) {
      console.log(`✨ 首次扫描，从部署区块开始:`);
      console.log(`   YT Vaults 起始区块: ${ytVaultsStartBlock}`);
      console.log(`   ytLP 起始区块: ${ytlpStartBlock}`);
      console.log(`   Lending 起始区块: ${lendingStartBlock}\n`);
    }
  }
```

```
175
176     // 1. 获取 YT 代币持有者
177     console.log("1. YT 代币持有者:");
178
179     for (const vault of YT_VAULTS) {
180       console.log(`  正在查询 ${vault.name} (${vault.address})...`);
181       const contract: Contract = new ethers.Contract(vault.address, ERC20_ABI,
    provider);
182       const filter = contract.filters.Transfer();
183       const events: (EventLog | Log)[] = await queryEventsInBatches(
184         contract,
185         filter,
186         ytVaultsStartBlock,
187         latestBlock
188       );
189
190       // 初始化该 vault 的地址集合（如果不存在）
191       if (!allYTAddresses.has(vault.address)) {
192         allYTAddresses.set(vault.address, new Set<string>());
193       }
194       const vaultAddresses = allYTAddresses.get(vault.address)!;
195
196       // 记录新增地址数量
197       const previousCount = vaultAddresses.size;
198
199       // 添加新发现的地址到全局集合
200       for (const event of events) {
201         if ("args" in event && event.args.to !== ethers.ZeroAddress) {
202           vaultAddresses.add(event.args.to as string);
203         }
204       }
205
206       const newAddressCount = vaultAddresses.size - previousCount;
207       if (newAddressCount > 0) {
208         console.log(`    发现 ${newAddressCount} 个新地址，总共追踪
    ${vaultAddresses.size} 个地址`);
209       }
210
211       // 查询所有曾经出现过的地址的当前余额
212       const holders: YTHolderData[] = [];
213       for (const address of vaultAddresses) {
214         const balance: bigint = await contract.balanceOf(address);
215         if (balance > 0n) {
216           holders.push({
217             address,
218             balance: ethers.formatEther(balance),
219           });
```

```
      }
    }

    // 按余额降序排序
    holders.sort((a, b) => parseFloat(b.balance) - parseFloat(a.balance));

    console.log(`  ${vault.name}: ${holders.length} 持有者`);
    if (holders.length > 0) {
      console.log(`  前 10 名持有者:`);
      const top10 = holders.slice(0, 10);
      top10.forEach((h: YTHolderData, index: number) =>
        console.log(`    ${index + 1}. ${h.address}: ${h.balance}`)
      );
    } else {
      console.log(`    暂无持有者`);
    }
  }

  // 2. 获取 LP 代币持有者
  console.log("\n2. LP 代币持有者 (ytLP):");
  console.log(`  正在查询 ytLP (${YTLP_ADDRESS})...`);
  const lpContract: Contract = new ethers.Contract(YTLP_ADDRESS, ERC20_ABI, provider);
  const lpFilter = lpContract.filters.Transfer();
  const lpEvents: (EventLog | Log)[] = await queryEventsInBatches(
    lpContract,
    lpFilter,
    ytlpStartBlock,
    latestBlock
  );

  // 记录新增地址数量
  const previousLPCount = allLPAddresses.size;

  // 添加新发现的地址到全局集合
  for (const event of lpEvents) {
    if ("args" in event && event.args.to !== ethers.ZeroAddress) {
      allLPAddresses.add(event.args.to as string);
    }
  }

  const newLPAddressCount = allLPAddresses.size - previousLPCount;
  if (newLPAddressCount > 0) {
    console.log(`  发现 ${newLPAddressCount} 个新地址，总共追踪 ${allLPAddresses.size} 个地址`);
  }

```

```
  // 查询所有曾经出现过的地址的当前余额
  const lpHolders: LPHolderData[] = [];
  const totalSupply: bigint = await lpContract.totalSupply();

  for (const address of allLPAddresses) {
    const balance: bigint = await lpContract.balanceOf(address);
    if (balance > 0n) {
      const share: string = (Number(balance) / Number(totalSupply) * 100).toFixed(4);
      lpHolders.push({
        address,
        balance: ethers.formatEther(balance),
        share: share + "%",
      });
    }
  }

  // 按余额降序排序
  lpHolders.sort((a, b) => parseFloat(b.balance) - parseFloat(a.balance));

  console.log(`  总计: ${lpHolders.length} 持有者`);
  if (lpHolders.length > 0) {
    console.log(`  前 10 名持有者:`);
    const top10 = lpHolders.slice(0, 10);
    top10.forEach((h: LPHolderData, index: number) =>
      console.log(`    ${index + 1}. ${h.address}: ${h.balance} (${h.share})`)
    );
  } else {
    console.log(`    暂无持有者`);
  }

  // 3. 获取 Lending 提供者
  console.log("\n3. Lending 提供者:");
  console.log(`  正在查询 Lending (${LENDING_ADDRESS})...`);
  const lendingContract: Contract = new ethers.Contract(LENDING_ADDRESS, LENDING_ABI, provider);
  const supplyFilter = lendingContract.filters.Supply();
  const supplyEvents: (EventLog | Log)[] = await queryEventsInBatches(
    lendingContract,
    supplyFilter,
    lendingStartBlock,
    latestBlock
  );

  // 记录新增地址数量
  const previousLendingCount = allLendingAddresses.size;

```

```
    // 添加新发现的地址到全局集合
    for (const event of supplyEvents) {
      if ("args" in event) {
        allLendingAddresses.add(event.args.dst as string);
      }
    }

    const newLendingAddressCount = allLendingAddresses.size - previousLendingCount;
    if (newLendingAddressCount > 0) {
      console.log(`  发现 ${newLendingAddressCount} 个新地址，总共追踪 ${allLendingAddresses.size} 个地址`);
    }

    // 查询所有曾经出现过的地址的当前余额
    const suppliers: LendingSupplierData[] = [];
    for (const address of allLendingAddresses) {
      const supplyBalance: bigint = await lendingContract.supplyBalanceOf(address);
      const borrowBalance: bigint = await lendingContract.borrowBalanceOf(address);

      if (supplyBalance > 0n || borrowBalance > 0n) {
        suppliers.push({
          address,
          supply: ethers.formatUnits(supplyBalance, 6),
          borrow: ethers.formatUnits(borrowBalance, 6),
          net: ethers.formatUnits(supplyBalance - borrowBalance, 6),
        });
      }
    }

    // 按净供应额降序排序
    suppliers.sort((a, b) => parseFloat(b.net) - parseFloat(a.net));

    console.log(`  总计: ${suppliers.length} 参与者`);
    if (suppliers.length > 0) {
      console.log(`  前 10 名参与者:`);
      const top10 = suppliers.slice(0, 10);
      top10.forEach((s: LendingSupplierData, index: number) =>
        console.log(
          `    ${index + 1}. ${s.address}: 供应=${s.supply} USDC, 借款=${s.borrow} USDC, 净额=${s.net} USDC`
        )
      );
    } else {
      console.log(`    暂无参与者`);
```

```
  }

  // 更新上次扫描的区块号
  lastScannedBlock = latestBlock;
  console.log(`\n📌 已记录扫描区块: ${lastScannedBlock}`);
}

// ==================== 执行 ====================

const POLL_INTERVAL_MS = 10000; // 10秒轮询间隔

async function main(): Promise<void> {
  const provider: JsonRpcProvider = new ethers.JsonRpcProvider(RPC_URL);

  console.log("=== ytLp 协议 Holder 数据监控 ===\n");
  console.log(`轮询间隔: ${POLL_INTERVAL_MS / 1000} 秒\n`);

  try {
    // 首次扫描: 从部署区块到当前区块
    console.log("📊 开始首次扫描...\n");
    const startTime = Date.now();
    await getAllHolders(provider, undefined, undefined, true);
    const endTime = Date.now();
    const duration = ((endTime - startTime) / 1000).toFixed(2);
    console.log(`\n✓ 首次扫描完成，耗时 ${duration} 秒`);

    // 启动轮询
    console.log(`\n⏰ 开始轮询，每 ${POLL_INTERVAL_MS / 1000} 秒检查一次新区块...\n`);

    setInterval(async () => {
      try {
        // 如果正在扫描，跳过本次轮询
        if (isScanning) {
          console.log(`⏰ [${new Date().toLocaleString()}] 跳过本次轮询（上次扫描仍在进行中）`);
          return;
        }

        const currentBlock = await provider.getBlockNumber();

        // 如果有新区块，进行增量扫描
        if (currentBlock > lastScannedBlock) {
          isScanning = true; // 标记开始扫描

          console.log(`\n${"=".repeat(60)}`);
          console.log(`⏰ [${new Date().toLocaleString()}] 发现新区块`);
```

```
            console.log(`${"=".repeat(60)}`);

            const scanStart = Date.now();
            await getAllHolders(provider, lastScannedBlock + 1, currentBlock, false);
            const scanDuration = ((Date.now() - scanStart) / 1000).toFixed(2);
            console.log(`\n✓ 增量扫描完成，耗时 ${scanDuration} 秒`);

            isScanning = false; // 标记扫描完成
          } else {
            console.log(`⏰ [${new Date().toLocaleString()}] 暂无新区块 (当前: ${currentBlock})`);
          }
        } catch (error) {
          console.error(`\n✗ 轮询过程中发生错误:`, error);
          isScanning = false; // 发生错误时也要重置标记
        }
      }, POLL_INTERVAL_MS);

    } catch (error) {
      console.error("\n✗ 发生错误:", error);
      process.exit(1);
    }
  }

  main();
```